Lenguaje Python

Cursada 2025

Más sobre la POO en Python

Conceptos claves: objetos y clases

- La **clase** define las variables de instancia y los métodos.Permite agrupasr objetos con características similares.
- Los **objetos** son instancias de una clase.
- Cuando se crea un objeto, se ejecuta automáticamente el método **__init()__** que permite inicializar el objeto.
- La definición de la clase especifica qué partes son públicas y **cuáles vamos a considerar no públicas**.

¿Cómo se especifica privado o público en Python?

Mensajes y métodos

TODO el procesamiento en este modelo es activado por mensajes entre objetos.

- El **mensaje** es el modo de comunicación entre los objetos. Cuando se invoca una función de un objeto, lo que se está haciendo es **enviando un mensaje** a dicho objeto.
- El **método** es la función que está asociada a un objeto determinado y cuya ejecución sólo puede desencadenarse a través del envío de un mensaje recibido.

¿@property?

> @property permite crear **propiedades** de una clase.

- Se aplica la función **property()** cuya forma general es:

```
property(fget=None, fset=None, fdel=None, doc=None)
```

¿En qué casos podemos usar propiedades?

Ejemplo

In [103…

```
class Demo:
    def __init__(self):
        self._x = 0

    def get_x(self):
        print('Estoy en get')
        return self._x

    def set_x(self, value):
        print('Estoy en set')
        self._x = value

    x = property(get_x, set_x)
```

In [104…

```
obj = Demo()
print(obj.x)
```

```
Estoy en get
0
```

O en su forma equivalente ...

In [106…

```
class Demo:
    def __init__(self):
        self._x = 0
    @property
    def x(self):
        print('Estoy en get')
        return self._x

    @x.setter
    def x(self, value):
        print('Estoy en set')
        self._x = value
```

In [107…

```
obj = Demo()
print(obj.x)
```

```
Estoy en get
0
```

- @property es un **decorador**.

¿Qué es un decorador?

> Un **decorador es una función** que recibe una función como argumento y **extiende** el comportamiento de esta última función sin modificarla explícitamente.

Hay otros decoradores

Métodos de clase

¿A qué creen que hacen referencia?

> Corresponden a los mensajes que se envían a la **clase**, no a las instancias de la misma.

- Se utiliza el decorador **@classmethod**.
- Se usa **cls** en vez de **self**. ¿A qué hace referencia este argumento?

In [ ]:

```
class Band():
    """ Define la entidad que representa a una banda   ..   """
    all_genres = set()

    @classmethod
    def clean_genres(cls, confirm=False):
        if confirm:
            cls.all_genres = set()

    def __init__(self, name, genres="rock"):
```

```
        self.name = name
        self.genres = genres
        self._members = []
        Band.all_genres.add(genres)

    def add_member(self, new_member):
        self._members.append(new_member)
```

In [ ]:
```
soda = Band("Soda Stereo")
nompa = Band("Nonpalidece", genres="reggae")
```

In [ ]:
```
Band.all_genres
```

In [ ]:
```
Band.clean_genres(True)
```

Hablemos de herencia

- Es uno de los conceptos más importantes de la POO.
- La herencia permite que una clase pueda *heredar* los atributos y métodos de otra clase, que se **agregan** a los propios.
- Este concepto permite sumar, es decir **extender** una clase.
- La clase que hereda se denomina **clase derivada** y la clase de la cual se deriva se denomina **clase base**.

Observemos estas clases

In [108…
```
class Vehículo:
    def __init__(self, cantidad_puertas, cantidad_ruedas, motor='Diesel'):
        self.cant_puertas = cantidad_puertas
        self.cant_ruedas = cantidad_ruedas
        self.motor = motor

    def __str__(self):
        return (f'Motor: {self.motor}, cantidad puertas: {self.cant_puertas}, cantidad ruedas: {self.cant_ruedas}')
```

In [109…
```
class Automóvil(Vehículo):
    def __init__(self, marca, modelo, patente, puertas, ruedas, motor):
        Vehículo.__init__(self, puertas, ruedas, motor)
        self.marca = marca
        self.modelo = modelo
        self._patente = patente
```

```python
    @property
    def patente(self):
        return self._patente

    @patente.setter
    def patente(self, nueva_patente):
        self._patente = nueva_patente
```

- ¿Cuál es la clase base? ¿Y la clase derivada? ¿Cuáles son las variables de instancia de un objeto Automóvil?
- ¿Por qué invocamos a **Vehículo__init__()**? ¿Qué pasa si no hacemos esto?

In [110…

```python
auto_nuevo = Automóvil('FIAT', "Cronos", 'AA588ZM', 5, 4, 'Nafta')
print(auto_nuevo)
```

```
Motor: Nafta, cantidad puertas: 5, cantidad ruedas: 4
```

Si trabajamos con vehículos, también podemos definir ...

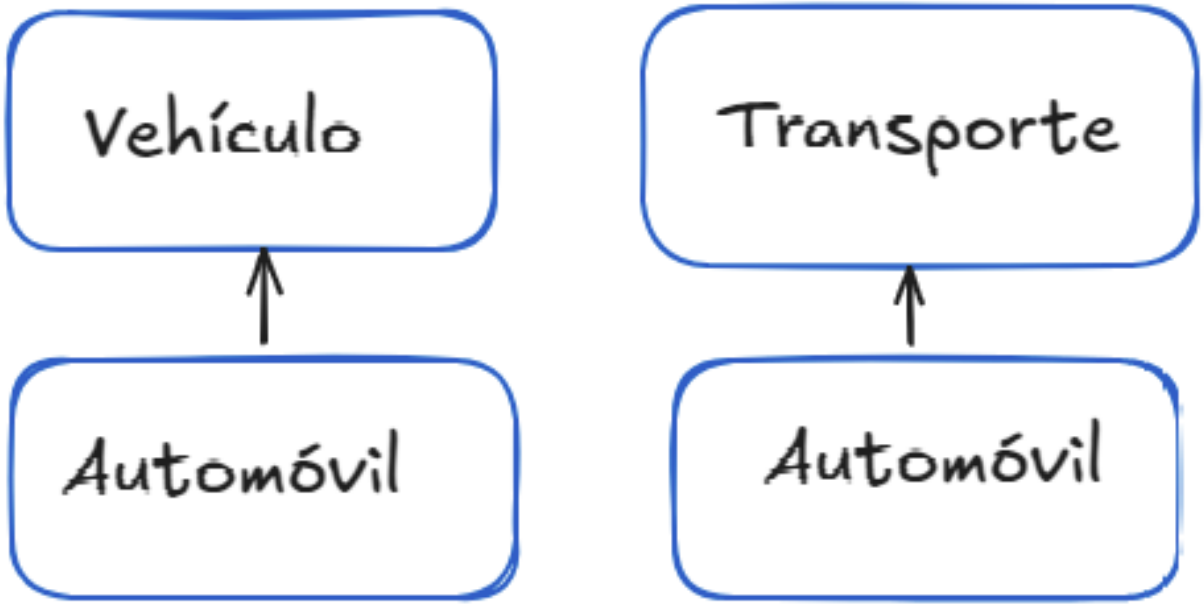
In [111…

```python
class Transporte():
    def __init__(self, max_pasajeros, tipo='Público'):
        self.tipo = tipo
        self.cant_max_pasajero = max_pasajeros

    def __str__(self):
        return (f'Tipo de transporte: {self.tipo}')
```

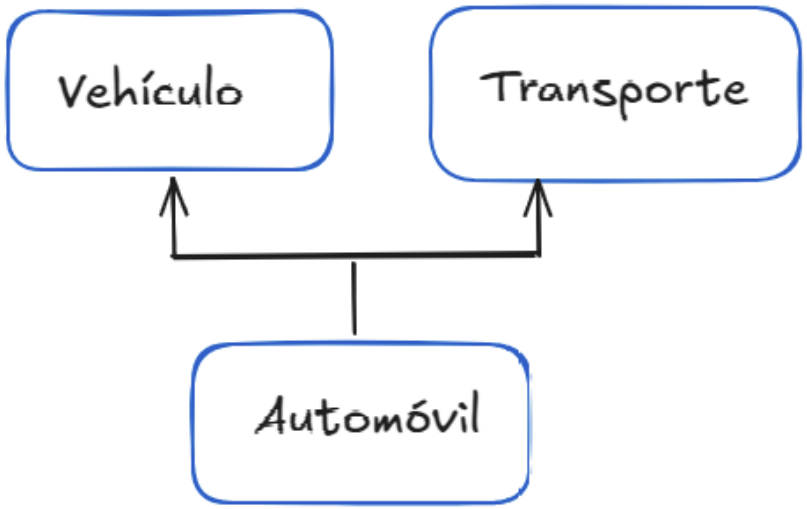
Un Auto es un vehículo, pero también es un medio de transporte ...

Podríamos pensar en algo así:



Aunque en Python podemos hacer algo mejor ...

Python tiene herencia múltiple



- Un automóvil es **"es un"** vehículo, pero también **es un** transporte.

En Python ...

In [112…
```
class Automóvil(Vehículo, Transporte):
    def __init__(self, marca, modelo, patente, tipo='Particular', pasajeros=0, puertas=4, ruedas=4, motor=None):
        Vehículo.__init__(self, puertas, ruedas, motor)
        Transporte.__init__(self, pasajeros, tipo)
        self.marca = marca
        self.modelo = modelo
        self._patente = patente

    @property
    def patente(self):
        return self._patente

    @patente.setter
    def patente(self, nueva_patente):
        self._patente = nueva_patente

#    def __str__(self):
#        return (f'Motor {self.motor}, cantidad puertas: {self.cant_puertas}, cantidad ruedas: {self.cant_ruedas}')
```

In [113…
```
auto_nuevo = Automóvil('FIAT', "Cronos", 'AA588ZM', 'Particular', puertas=4, motor='Nafta')
print(auto_nuevo)
```

```
Motor: Nafta, cantidad puertas: 4, cantidad ruedas: 4
```

A tener en cuenta ...

- MRO "Method Resolution Order"
- Por lo tanto, es MUY importante el orden en que se especifican las clases bases.
- Más información en documentación oficial

In [114…
```
Automóvil.__mro__
```

Out[114]:
```
(__main__.Automóvil, __main__.Vehículo, __main__.Transporte, object)
```

In [115…
```
class Automóvil(Transporte, Vehículo):
    def __init__(self, marca, modelo, patente, tipo='Particular', pasajeros=0, puertas=4, ruedas=4, motor=None):
```

```
        Vehiculo.__init__(self, puertas, ruedas, motor)
        Transporte.__init__(self, pasajeros, tipo)
        self.marca = marca
        self.modelo = modelo
        self._patente = patente

    @property
    def patente(self):
        return self._patente

    @patente.setter
    def patente(self, nueva_patente):
        self._patente = nueva_patente
```

In [116…

```
Automóvil.__mro__
```

Out[116]:

```
(__main__.Automóvil, __main__.Transporte, __main__.Vehículo, object)
```

DEASAFíO 1

> ¿Cuál es el MRO de D?

In [120…

```
class A():
    pass
class B(A):
    pass
class C(A):
    pass
class D(B, C):
    pass
```

In [121…

```
D.__mro__
```

Out[121]:

```
(__main__.D, __main__.B, __main__.C, __main__.A, object)
```

Hagamos el siguiente cambio

In [ ]:

```
class Vehículo:
    def __init__(self, cantidad_puertas, cantidad_ruedas, motor='Diesel'):
        self.cant_puertas = cantidad_puertas
```

```
        self.cant_ruedas = cantidad_ruedas
        self.motor = motor
        self.tipo = 'Sedan'
```

In [ ]:
```
class Transporte():
    def __init__(self, max_pasajeros, tipo='Público'):
        self.tipo = tipo
        self.cant_max_pasajero = max_pasajeros

    def __str__(self):
        return (f'Tipo de transporte: {self.tipo}')
```

¿Cuál es el problema?

In [ ]:
```
class Automóvil(Transporte, Vehículo):
    def __init__(self, marca, modelo, patente, tipo='Particular', pasajeros=0, puertas=4, ruedas=4, motor=None):
        Transporte.__init__(self, pasajeros, tipo)
        Vehículo.__init__(self, puertas, ruedas, motor)
        self.marca = marca
        self.modelo = modelo
        self._patente = patente
    def __str__(self):
        return (f'El {self.marca} {self.modelo} es un {self.tipo} de {self.cant_puertas} puertas')
```

In [ ]:
```
auto_nuevo = Automóvil('FIAT', "Cronos", 'AA588ZM', 'Particular', puertas=4, motor='Nafta')
print(auto_nuevo)
```

¿Qué tenemos que tener en cuenta?

- El orden de las clases bases en la definición de la clase.
- El orden de la invocación de los __init__.

¿Qué términos asociamos con la programación orientada a objetos?

Destacados ...

- Encapsulamiento

    - **class**, métodos privados y públicos, propiedades.
- Herencia
    - Clases bases y derivadas.
    - Herencia múltiple.
- **¿Alguno más?**

Polimorfismo

- Capacidad de los objetos de distintas clases de responder a mensajes con el mismo nombre.
- Ejemplo: + entre enteros y cadenas.

In [ ]:
```
print("hola " + "que tal.")
print(3 + 4)
```

Observemos este código

In [ ]:
```
class Musician:
    def __init__(self, name, instrument=None, band=None):
        self.name = name
        self.has_a_band = band!=None
        self._band = band
        self.instrument = instrument

    @property
    def band(self):
        if self.has_a_band:
            return self._band
        else:
            return None

    @band.setter
    def band(self, new_band):
        self._band = new_band
        self.has_a_band = self._band!=None
```

¿Podemos sumar dos músicos?

In [ ]:
```
adele = Musician("Adele")
sting = Musician("Sting", "The Police")

print(adele + sting)
```

No podemos sumar dos músicos, salvo que ...

In [ ]:
```
class Musician:
    def __init__(self, name, instrument=None, band=None):
        self.name = name
        self.has_a_band = band!=None
        self._band = band
        self.instrument = instrument

    def __add__(self, other):
        return (f"Nuevo dúo: {self.name} y {other.name}")

    @property
    def band(self):
        if self.has_a_band:
            return self._band
        else:
            return None

    @band.setter
    def band(self, new_band):
        self._band = new_band
        self.has_a_band= self._band!=None
```

In [ ]:
```
adele = Musician("Adele")
sting = Musician("Sting", "The Police")

print(adele + sting)
```

Creamos iteradores

> Un **iterador** es un objeto que permite recorrer **uno a uno** los elementos de una estructura de datos para poder operar con ellos.

- Un objeto iterable tiene que implementar un método **__next__** que debe devolver los elementos, **de a uno por vez**, comenzando por el primero.
- Y al llegar al final de la estructura, debe levantar la excepción **StopIteration**.

- Todas implementan un método especial denominado **__iter__**.
  - **__iter__** devuelve un iterador capaz de recorrer la secuencia.

Los siguientes códigos son equivalentes:

In [ ]:
```
lista = ['uno', 'dos', 'tres']
for palabra in lista:
    print(palabra)
```

In [ ]:
```
iterador = iter(lista)
while True:
    try:
        palabra = next(iterador) # o iterador.__next__()
    except StopIteration:
        break
    print(palabra)
```

- La función **iter** retorna un objeto iterador.

¿Son equivalentes?

In [ ]:
```
for elem in lista:
    print (elem)
```

In [ ]:
```
for elem in iterador:
    print (elem)
```

Veamos este ejemplo:

In [ ]:
```
class CadenaInvertida:
    def __init__(self, cadena):
        self._cadena = cadena
        self._posicion = len(cadena)
```

```
    def __iter__(self):
        return(self)

    def __next__(self):
        if self._posicion == 0:
            raise(StopIteration)
        self._posicion = self._posicion - 1
        return(self._cadena[self._posicion])
```

In [ ]:
```
cadena_invertida = CadenaInvertida('La clase próxima empezamos con pandas')

for caracter in cadena_invertida:
     print(caracter, end=' ')
```

- ¿Qué creen que imprime?

DESAFIO 2

> Implementar la clase **CadenaCodificada**, que dada una cadena de caracteres, me permita trabajar con la misma en forma codificada, según la codifcación Cesar (vista en las primeras clases). Podemos recorrer con un **for** un objeto de clase **CadenaCodificada**, los cual permite acceder uno a uno a los caracteres codificados de la misma.

Nota: implementar este objeto como un objeto iterable.

In [ ]:
```
#Una posible solución
from functools import reduce

class CadenaCodificada:
    def __init__(self, cadena):
        self._cadena = cadena
        self._posicion = 0

    def __iter__(self):
        return(self)

    def __next__(self):
        if self._posicion == len(self._cadena):
            raise(StopIteration)
        car = self._cadena[self._posicion]
        self._posicion = self._posicion + 1
```

```
        return(chr(ord(car) + 1))

    def __str__(self):
        lista = map(lambda c : chr(ord(c) + 1), self._cadena)
        return reduce(lambda c1, c2: c1 + c2, lista)
```

Ejemplo de uso de una CadenaCodificada

In [ ]:
```
mi_cadena = CadenaCodificada("Hola")

for caracter in mi_cadena:
     print(caracter, end=" ")
```

In [ ]:
```
print(mi_cadena)
```

CONSIGNA: agregar un método que permita obtener la cadena original

CONSIGNA: probamos en casa

¿Qué podemos decir de las variables de instancias cuyo nombre comienza con __?

In [ ]:
```
class A:
    def __init__(self, x, y, z):
        self.varX = x
        self._varY = y
        self.__varZ = z

    def demo(self):
        return f"ESTOY en A: x: {self.varX} -- y:{self._varY} --- z:{self.__varZ}"

class B(A):
    def __init__(self):
        A.__init__(self, "x", "y", "z")

    def demo(self):
        return f"ESTOY en B: x: {self.varX} -- y:{self._varY} --- z:{self.__varZ}"
```

In [ ]:
```
objB = B()
print(objB.demo())
```

Para los que quieran seguir un poco más ...

- https://realpython.com/python-classes/
- https://realpython.com/inheritance-composition-python/
- https://realpython.com/instance-class-and-static-methods-demystified/

Seguimos la próxima ...